Experiment 11: Implementation of the VFDT Algorithm for Data Classification

Aim

To implement the Very Fast Decision Tree (VFDT) algorithm, which builds a decision tree incrementally from a data stream using the Hoeffding bound to decide when to split, covering the data classification process, the Hoeffding bound, incremental decision tree construction and the data pipeline.

Theory

The Very Fast Decision Tree, also called the Hoeffding Tree, builds a decision tree from a data stream without storing the stream. Each example is read once, used to update the statistics stored at a leaf, and then discarded. Only counts are kept in memory, so the memory requirement does not grow with the length of the stream.

The key idea is the Hoeffding bound. It states that after n independent observations of a variable with range R, the true mean differs from the observed mean by at most epsilon, with probability 1 - delta:

```
epsilon = sqrt( (R^2 * ln(1 / delta)) / (2 * n) )
```

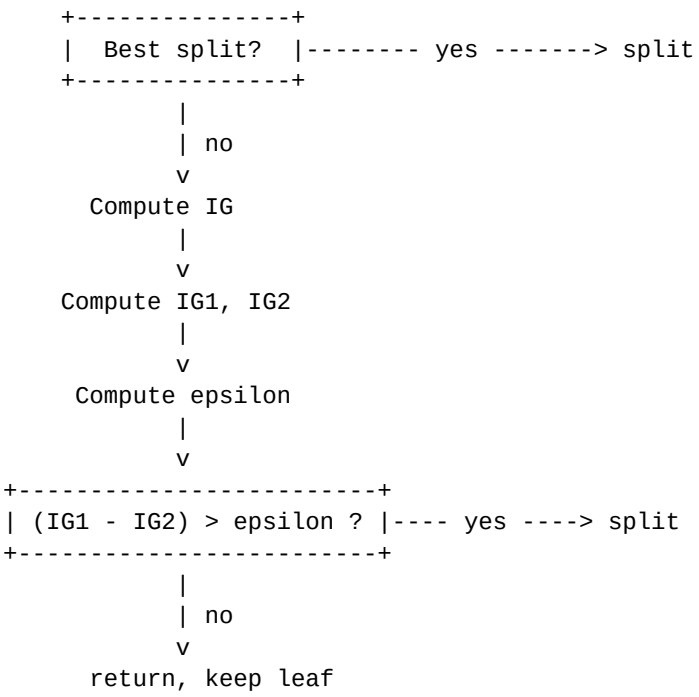
At a leaf, the information gain of every attribute is calculated. If the gap between the best attribute and the second best attribute is greater than epsilon, then the best attribute is confidently the winner and the leaf is split. If the gap is smaller, more examples are collected before deciding. This allows the tree to grow from a small number of examples with a statistical guarantee, instead of scanning the whole dataset as a traditional decision tree does.

Data Pipeline

```
Data stream
     |
     v
Data collection
     |
     v
Data preprocessing
     |
     v
Feature extraction
     |
     v
VFDT learning process
     |
     v
Attribute statistics calculation
     |
     v
Hoeffding bound calculation
     |
     v
Best split selection
     |
     v
Decision tree update
     |
```

```
     v
Prediction / classification
     |
     v
Model monitoring
```

Split Decision Flow



Implementation Steps

1. Library
2. VFDT Node
3. VFDT Tree
4. Entropy
5. Hoeffding bound
6. Predict
7. Dataset
8. Discretize
9. Accuracy calculation

Requirements

- Python 3.x
- Jupyter Notebook
- NumPy
- Scikit-learn

Step 1: Import Libraries

```
import numpy as np
import math
from collections import defaultdict
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Interpretation: Imports the modules needed to generate the dataset, store the counts, compute the bound and measure the accuracy.

Step 2: VFDT Node

```
class VFDTNode:

    def __init__(self):
        self.is_leaf = True
        self.children = {}
        self.split_feature = None
        self.class_counts = defaultdict(int)
        self.feature_counts = defaultdict(lambda: defaultdict(int))
        self.samples = 0
```

Interpretation: Every node starts as a leaf with no children. class_counts stores how many samples of each class reached the node, feature_counts stores how often each (value, label) pair was seen for every feature, and samples counts the total examples that reached the node.

Step 3: VFDT Tree

```
class VFDT:

    def __init__(self, delta=0.01, min_samples=50):
        self.root = VFDTNode()
        self.delta = delta
        self.min_samples = min_samples
        self.num_features = None
```

Interpretation: Creates the root leaf and stores the confidence parameter delta and the minimum number of samples a leaf must see before a split is attempted.

Step 4: Entropy and Information Gain

```
    def entropy(self, class_counts):

        total = sum(class_counts.values())

        if total == 0:
            return 0

        entropy = 0

        for c in class_counts.values():
            p = c / total
            if p > 0:
                entropy -= p * math.log2(p)

        return entropy


    def information_gain(self, node, feature):

        total_entropy = self.entropy(node.class_counts)

        feature_values = defaultdict(lambda: defaultdict(int))
        total = node.samples

        for (value, label), count in node.feature_counts[feature].items():
            feature_values[value][label] += count
```

```
        weighted_entropy = 0

        for value, counts in feature_values.items():
            sub_total = sum(counts.values())
            weighted_entropy += (sub_total / total) * self.entropy(counts)

        return total_entropy - weighted_entropy
```

Interpretation: entropy() computes the impurity of a set of class counts. information_gain() returns the total entropy minus the weighted entropy of the subsets produced by splitting on a feature. Both work only from the counts stored at the node, so the original examples are never needed.

Step 5: Hoeffding Bound and Update

```
    def hoeffding_bound(self, R, delta, n):
        return math.sqrt((R * R * math.log(1 / delta)) / (2 * n))


    def update(self, x, y):

        if self.num_features is None:
            self.num_features = len(x)

        node = self.root

        while not node.is_leaf:
            feature = node.split_feature
            value = int(x[feature])
            if value not in node.children:
                node.children[value] = VFDTNode()
            node = node.children[value]

        node.samples += 1
        node.class_counts[y] += 1

        for f in range(self.num_features):
            value = int(x[f])
            node.feature_counts[f][(value, y)] += 1

        if node.samples >= self.min_samples:
            self.try_split(node)
```

Interpretation: update() routes one example from the root down to a leaf, creating a child node if the attribute value has not been seen before. It then updates the counts at that leaf and attempts a split once the leaf has enough samples.

Step 6: Try Split

```
    def try_split(self, node):

        gains = []

        for f in range(self.num_features):
            gain = self.information_gain(node, f)
            gains.append((gain, f))

        gains.sort(reverse=True)

        if len(gains) < 2:
            return
```

```
        best_gain, best_feature = gains[0]
        second_gain, _ = gains[1]

        epsilon = self.hoeffding_bound(1, self.delta, node.samples)

        if best_gain - second_gain > epsilon:
            node.is_leaf = False
            node.split_feature = best_feature
            node.children = {}
```

Interpretation: The gains of all attributes are sorted in descending order. If the gap between the best and the second best gain is greater than epsilon, the leaf becomes an internal node that splits on the winning attribute. Otherwise the leaf is left unchanged and waits for more data.

Step 7: Predict

```
    def predict_one(self, x):

        node = self.root

        while not node.is_leaf:
            feature = node.split_feature
            value = int(x[feature])
            if value not in node.children:
                break
            node = node.children[value]

        if len(node.class_counts) == 0:
            return 0

        return max(node.class_counts, key=node.class_counts.get)


    def predict(self, X):

        predictions = []

        for row in X:
            predictions.append(self.predict_one(row))

        return predictions
```

Interpretation: predict_one() walks the tree until it reaches a leaf or an unseen attribute value, then returns the majority class of that node. predict() applies this to every row of the test set.

Step 8: Dataset

```
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    random_state=42
)
```

Interpretation: Generates 1000 samples with 4 informative features and no redundant features.

Step 9: Discretize

```
X = np.round(X)
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=5
)
```

Interpretation: VFDT splits on discrete attribute values, so the continuous features are rounded to integers. 70% of the data is used for training and 30% for testing.

Step 10: Train Incrementally

```
tree = VFDT(delta=0.01, min_samples=40)

for x, label in zip(X_train, y_train):
    tree.update(x, label)
```

Interpretation: The examples are fed one at a time, exactly as they would arrive in a stream. No example is stored, only the counts at each leaf.

Step 11: Accuracy Calculation

```
predictions = tree.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("Accuracy =", accuracy)
```

Interpretation: Measures how many test samples were classified correctly by the incrementally built tree.

Sample Input

No external input file is required. The data is generated internally:

```
Samples              : 1000
Features             : 4
Informative features : 4
Redundant features   : 0
random_state         : 42
Train / test split   : 70% / 30%
delta                : 0.01
min_samples          : 40
```

Expected Output

```
Dataset shape : (1000, 4)
Training samples : 700
Testing samples  : 300
Root split feature : 1
Number of children : 10
Accuracy = 0.76
```

Interpretation

- 1000 samples with 4 informative features are generated.
- The features are rounded so that the tree can split on discrete values.
- Each training example is processed once and then discarded.
- A leaf tries to split only after it has seen at least 40 samples.
- The Hoeffding bound decides whether the best attribute is a confident winner.
- The tree split the root on feature 1 and created 10 children, one per attribute value.
- The accuracy obtained on the test data is approximately 0.76.

Conclusion

- The VFDT (Very Fast Decision Tree) algorithm was successfully implemented using Python.
- It efficiently classified streaming data by constructing the decision tree incrementally.
- The Hoeffding bound helped in selecting the best split without processing the entire dataset.
- The model required less memory and processed large datasets quickly.